

train-test split (of $data[:, 1:]$, $data[:, 0]$) test-size

Feature → input data matrix
: → all row
1: → col index 1 2 3 4 5
→ 2 3 4 5 6 7 8 9 10

Target (output)
: → all row
0 → 1st col

test-size = 0.2 → test set = 20%
train = 80%

Model learn -
Model test -

```
scaler = StandardScaler()  
X_train = scaler.fit_transform(X_train)  
X_test = scaler.transform(X_test)
```

Scaling

fit → means std calculate
transform → fit to data

transform → just scale

test data 7276
mean 270 71

random value initialize
2(0, 1)

X.shape → (row, col)
[0] [1]

col feature to (237) 24th

more accurate calc
gradient calc

model vul
260 7208 measure
40

```
class MySimpleNN():  
    def __init__(self, X):  
        self.weights = torch.rand(X.shape[1], 1, dtype=torch.float64, requires_grad=True)  
        self.bias = torch.zeros(1, dtype=torch.float64, requires_grad=True)  
  
    def forward(self, X):  
        z = torch.matmul(X, self.weights) + self.bias  
        y_pred = torch.sigmoid(z)  
        return y_pred  
  
    def loss_function(self, y_pred, y):  
        # Clamp predictions to avoid log(0)  
        epsilon = 1e-7  
        y_pred = torch.clamp(y_pred, epsilon, 1 - epsilon)  
        # Calculate loss  
        loss = -(y_train_tensor * torch.log(y_pred) + (1 - y_train_tensor) * torch.log(1 - y_pred)).mean()  
        return loss
```

0 initialize

$z = Wx + b$
0.0000001

y-pred (0.999999)
range 0 to 1
237 24th
237 24th

Loss = $-[y \log(y_pred) + (1 - y) \log(1 - y_pred)]$

nn.Module
parent class
initialize

input feature
1 layer

```
class MySimpleNN(nn.Module):  
    def __init__(self, num_features):  
        super().__init__()  
        self.linear = nn.Linear(num_features, 1)  
        self.sigmoid = nn.Sigmoid()  
  
    def forward(self, features):  
        out = self.linear(features)  
        out = self.sigmoid(out)  
        return out
```

binary classification

single output

out → output → weighted S

lines,

```
#create model
model = MySimpleNN(X_train_tensor.shape[1])

#optimizer
optimizer = torch.optim.SGD(model.parameters(), lr = learning_rate)

#define loop
for epoch in range(epochs):

    #forward pass
    y_pred = model(X_train_tensor)

    #Loss calculate
    loss = loss_function(y_pred,y_train_tensor.reshape(-1,1))

    #zero gradient
    optimizer.zero_grad()

    #backward pass
    loss.backward()

    #parameter update
    optimizer.step()

#print loss in each epoch
print(f" {epoch+1} , Loss = {loss.item()} ")
```

→ stochastic Gradient Descent

↓
update
হলো

↖
update
হলো

dog, cat, bird
↑
training label (২৭০৫)
↑ class
↑ for

fit

```
encoder= LabelEncoder()
y_train= encoder.fit_transform(y_train)
y_test= encoder.transform(y_test)
```

string → num

Transform

class ২৭০৫
Num → convert করে
dog - 0
cat - 1
bird - 2

Example

ধরা:

```
<> Python
y_train = ["yes", "no", "yes"]
y_test = ["no", "yes"]
```

encode করার পর:

```
y_train = [1, 0, 1]
y_test = [0, 1]
```

→ All
→ unique

২৭২৫